DWEC — Unitat 2: Activitat Puntuable

Problema dels Graons (Implementació Iterativa i Recursiva en JavaScript)

Alumne	Noé Conde Vila
Curs	2n DAW (Semipresencial) — IES Maria Enríquez
Assignatura	DWEC — Desenvolupament Web en Entorn Client
Data	Setembre 2026

1. Enunciat de l'Activitat

Imagina que una persona està a la planta de lloguers d'un edifici i vol arribar a la teulada. L'edifici té n graons numerats de 1 a n. A cada pas, la persona pot fer un salt de 1 graó o un salt de 2 graons.

Escriu una funció amb una implementació iterativa i una altra implementació recursiva en JavaScript que, donat un nombre enter positiu n, retorne el nombre de maneres diferents de pujar des del sòl al graó n.

Recomanacions i Exemples:

- **Gestionar correctament n = 1 i n = 2.**
- **Exemple per a n = 3:** Les maneres són (1+1+1), (1+2), (2+1) => **3 maneres**.
- **Exemple per a n = 4:** Les maneres són (1+1+1+1), (1+1+2), (1+2+1), (2+1+1), (2+2) => **5 maneres**.
- **Exemples d'execució esperats:**
  - `comptaManeres(1)` -> 1
  - `comptaManeres(2)` -> 2
  - `comptaManeres(3)` -> 3
  - `comptaManeres(4)` -> 5

2. Implementació en JavaScript (Codi)

2.1. Implementació Iterativa (`comptaManeresIteratiu`)

```
function comptaManeresIteratiu(n) {
  if (n <= 0) return 0;
  if (n === 1) return 1;
  if (n === 2) return 2;

  let anterior = 1;
  let actual = 2;

  for (let i = 3; i <= n; i++) {
    let seguent = anterior + actual;
    anterior = actual;
    actual = seguent;
  }
  return actual;
}

// Proves
console.log("Iteratiu n=1 ->", comptaManeresIteratiu(1));
console.log("Iteratiu n=2 ->", comptaManeresIteratiu(2));
console.log("Iteratiu n=3 ->", comptaManeresIteratiu(3));
console.log("Iteratiu n=4 ->", comptaManeresIteratiu(4));
console.log("Iteratiu n=5 ->", comptaManeresIteratiu(5));
```

2.2. Implementació Recursiva (`comptaManeresRecursiu`)

```
function comptaManeresRecursiu(n) {
  if (n <= 0) return 0;
  if (n === 1) return 1;
  if (n === 2) return 2;

  return comptaManeresRecursiu(n - 1) + comptaManeresRecursiu(n - 2);

}

// Proves
console.log("Recursiu n=1 ->", comptaManeresRecursiu(1));
console.log("Recursiu n=2 ->", comptaManeresRecursiu(2));
console.log("Recursiu n=3 ->", comptaManeresRecursiu(3));
console.log("Recursiu n=4 ->", comptaManeresRecursiu(4));
console.log("Recursiu n=5 ->", comptaManeresRecursiu(5));
```

---

3. Preguntes de Comprensió i Justificació

1. Explica, pas a pas, com arriba la solució iterativa al resultat per a n qualsevol. Detalla quines són les dues últimes quantitats que es mantenen i com es van actualitzant en el bucle.

Se empieza comprobando los primeros casos posibles, si es 1 o 2, devuelve el mismo valor, para cualquier otro se usa el bucle que calcula escalón a escalón guardando los dos ultimos valores calculados sumandolos para obtener el siguiente hasta llegar a 'n'.

---

2. Per al cas recursiu, descriu la relació de recurrència que s'està utilitzant i identifica els casos base. Explica per què la solució és correcta.

Este metodo usa el patrón de recurrencia Fibonacci, que suma los dos valores anteriores para obtener el siguiente valor, por lo que siempre tendra el valor de los dos ultimos escalones. f(n) = f(n-1) + f(n-2).

---

3. Què faries per gestionar casos extremadament grans de n (p. ex., n en milions) mantenint l'eficiència?

Tendriamos varios problemas, el tiempo de ejecución y la perdida de precisión en los calculos tan grandes.

Para no saturar la memoria con un bucle haciendo tanta recursividad, guardamos los valores en un objeto para no repetir calculos, el primero valor siempre sera 1 y el segundo 2, a partir de ahi sumamos y guardamos en el objeto como hemos hecho en este ejercicio.

Tambien usariamos BigInt para evitar la perdida de precision en los calculos tan grandes ya que number, let o const no soportan numeros tan grandes y perderiamos precision.